



PANDAS

Write By. 김지훈

Contents

01. Pandas 의 개념과 실행환경

02. Series 와 data frame

03. 데이터 호출과 정제작업(Cleaning)

04. 데이터 고급 기술

강의 목표

PANDAS 의 개념과 활용

Pandas 기본

PANDAS 의 기본 개념과
실행 환경을 구성한다.

Pandas 활용

Pandas 를 활용한 데이터
정제작업 과정을 알아본다.

Pandas 고급 기술

Pivot 과 group by 등의
기능들에 대해서 알아본다.

01.

Pandas 의 개념과 실행환경

Pandas 의 개념과 실행환경

Pandas 란?

- PANDAS 는 데이터 전처리(정제) 및 분석을 위한 Python 라이브러리 이다.
- Series 와 Data Frame 이라는 데이터 구조를 활용한다.

```
pip install pandas
```

```
import pandas as pd
```

<- PANDAS 를 사용하기 위해서는 우선 설치 및 import 가 필요 하다.

Pandas 의 개념과 실행환경

Pandas 실행환경

- PANDAS 는 python 프로젝트에서 실행해야 한다.
- 우리는 online 환경은 google Col-lab 환경에서 실행할 예정이다.

Google Colab

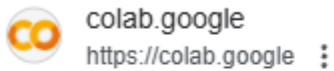
아무런 설치 없이 브라우저에서 **직접 파이썬(Python) 코드를 작성하고 실행**할 수 있게 해주는
구글의 **클라우드 기반 서비스**

Pandas 의 개념과 실행환경

☑ Pandas 실행환경

- (1번) google 검색에서 google colab 을 찾아서 들어간다.
- (2번) Open Colab 버튼을 눌러 colab 을 시작해 주자

1



Google Colab

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs.

Google Colaboratory

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

2

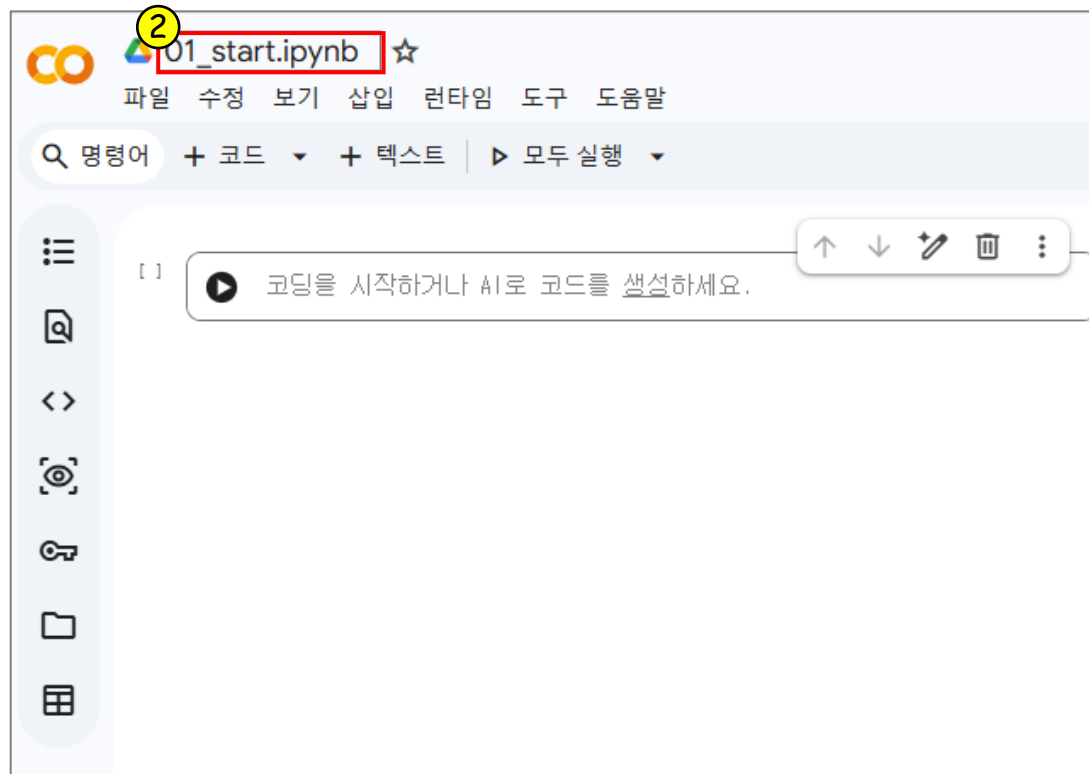
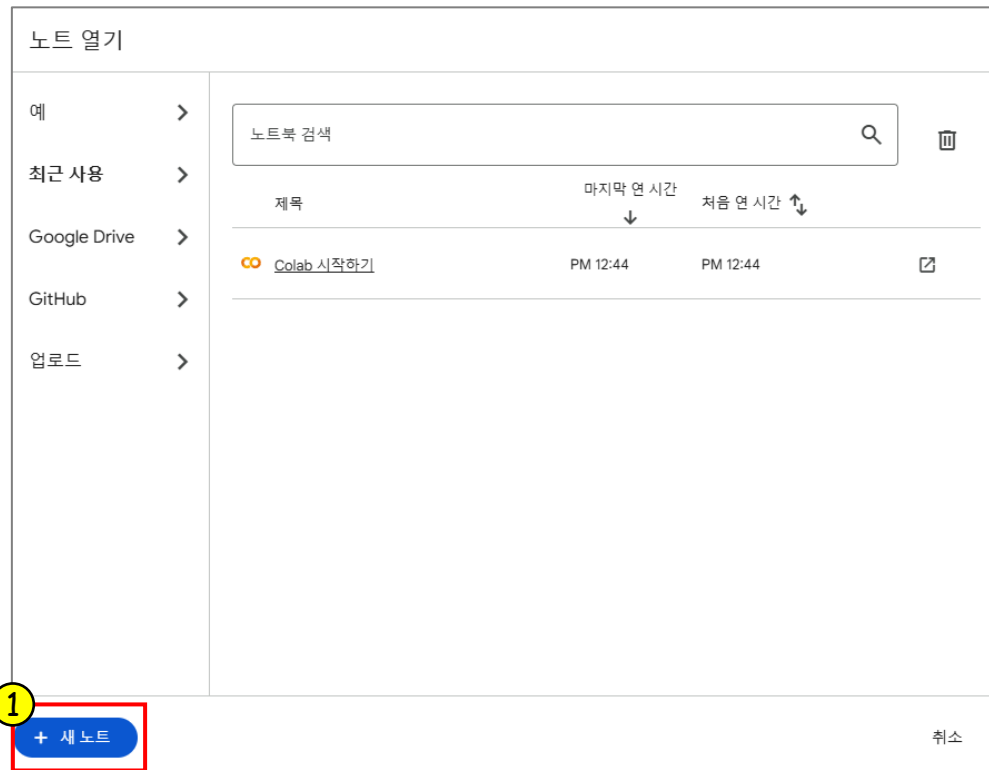
Open Colab

New Notebook

Pandas 의 개념과 실행환경

Pandas 실행환경

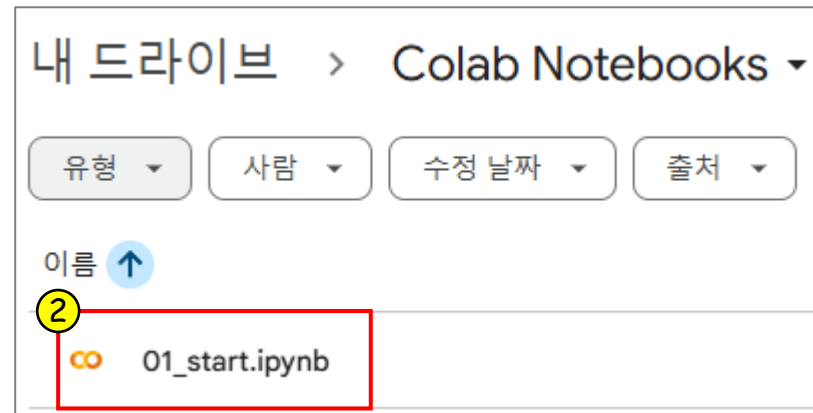
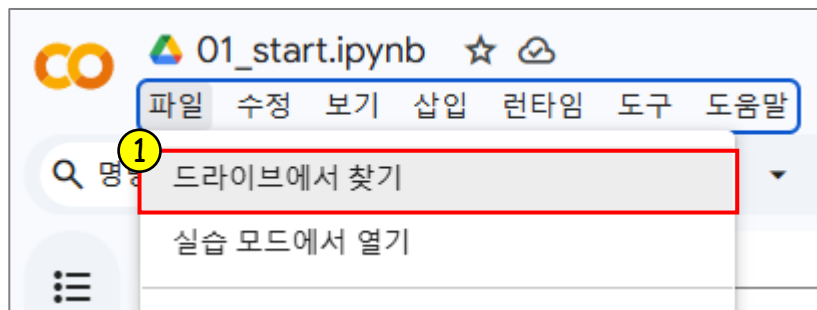
- (1번) 새노트 버튼을 클릭해 새로운 편집기를 생성한다.
- (2번) 이름을 01_start.ipynb 로 변경해 준다.



Pandas 의 개념과 실행환경

☑ Pandas 실행환경

- (1번)파일 > 드라이브에서 찾기를 선택해 보면
- (2번) 생성한 파일이 구글 드라이브의 **내드라이브 > Colab Notebooks** 에 저장되었음을 확인 할 수 있다.



Pandas 의 개념과 실행환경

☑ Pandas 실행환경

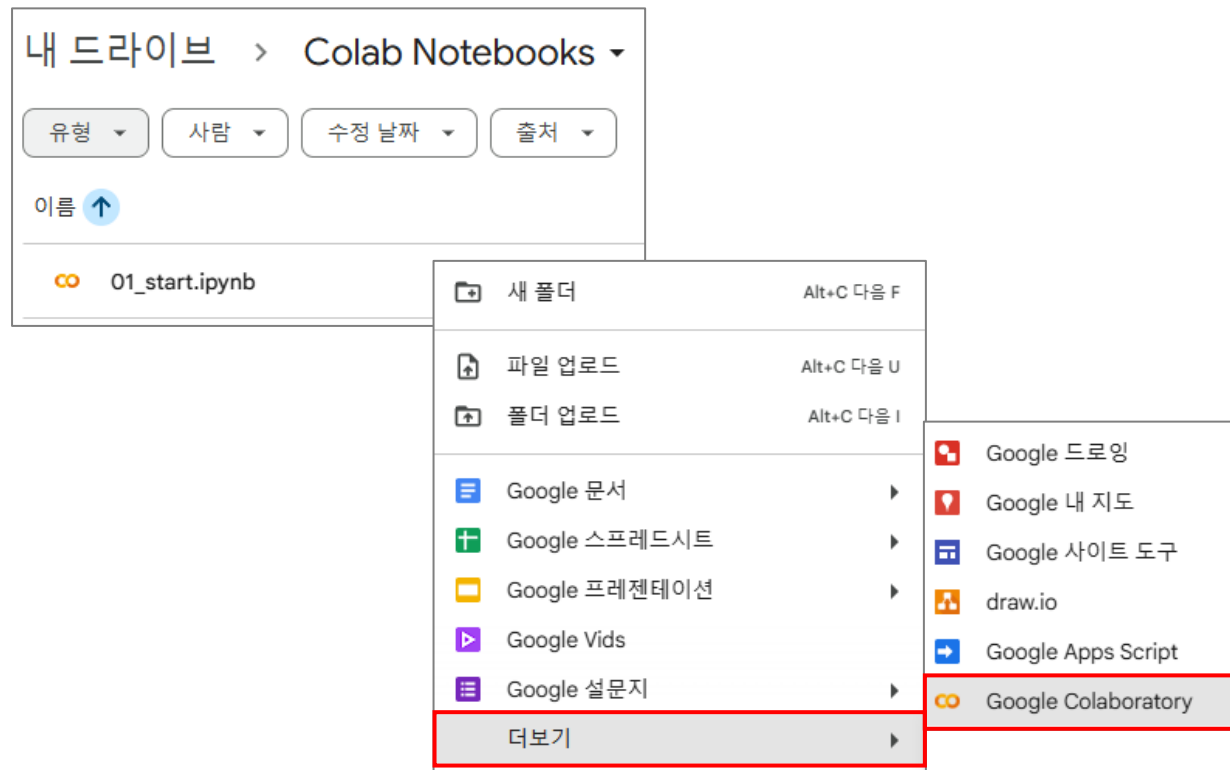
- Colab 에서 기본적인 내용들을 실행해 보자
- 파일 생성은 구글 드라이브 내에서 우클릭 > 더보기 > Google Colaboratory 로도 가능

```
import pandas as pd

mydata = {
    'cars': ['BMW', 'BENZ', 'Volvo', 'Ford'],
    'passings': [3, 2, 7, 4]
}

df = pd.DataFrame(mydata)
print(df)
```

실습파일 : 01_start.ipynb



02.

Series 와 data frame

Series 와 data frame

Series

- Serise 모든 데이터 타입을 수용하는 1차원 리스트와 같다.
- Index 는 기본 숫자로 되어있으나 문자로 지정 할 수도 있다.

index	value
0	A
1	B
2	C
3	D

index	value
Day1	420
Day2	380
Day3	390
Day4	380

```
import pandas as pd
```

```
a = [1,3,5,7]
```

```
se = pd.Series(a)
```

```
print(a)
```

```
print(se)
```

```
[1, 3, 5, 7]
```

```
0    1  
1    3  
2    5  
3    7  
dtype: int64
```

실습파일 : 02_series.ipynb

Series 와 data frame

Data frame

- Data frame 은 2차원 배열 형태의 데이터 이다.
- Series 가 합쳐진 모양인데, Excel 형태의 데이터라고 볼 수도 있다.

Calories series		Duration series		Score series	
0	420	0	50	0	100
1	380	1	40	1	70
2	390	2	45	2	80



	column			
	calories	duration	score	
0	420	50	100	row
1	380	40	70	
2	390	45	80	

```
data = {  
    'calories':[420,380,390],  
    'duration':[80,40,45],  
    'score':[700,80,70]  
}
```

실습파일 : 03_data_frame.ipynb

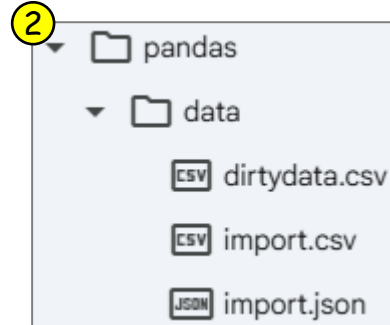
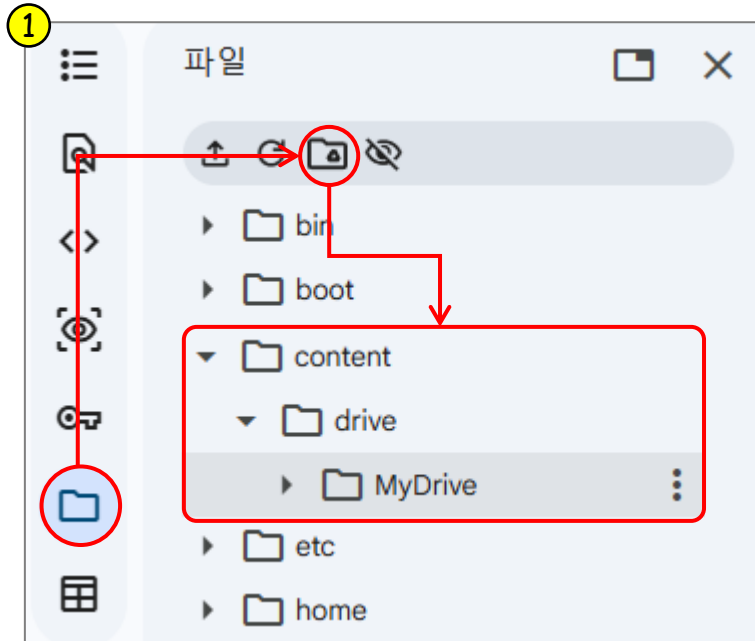
03.

데이터 호출과 정제작업

데이터호출과 정제작업

☑ 데이터호출

- COLAB 에서 google driver 의 데이터를 다루기 위해서는 해줘야 할 작업이 있다.
- (1번)Google driver 연동 후 content>drive>MyDrive 로 가면 내 드라이브와 연결이 되어있다.
- (2번) data 폴더에 공유된 3개의 파일을 업로드 하자



데이터호출과 정제작업

데이터호출

- 특정 폴더의 파일을 가져오거나 저장해 보자

```
import pandas as pd

print('import.csv 불러오기')
df = pd.read_csv('/content/drive/MyDrive/pandas/data/import.csv')
print(df)
```

```
result = {
    "Duration": [60, 60, 45, 45, 60, 65],
    "Pulse": [110, 117, 103, 109, 117, 102],
    "Calories": [409, 479, 340, 282, 406, 300]
}
df = pd.DataFrame(result)
df.to_csv('/content/drive/MyDrive/pandas/data/export.csv')
print('저장완료')
```

실습파일 : 04_import_data.ipynb

데이터 호출과 정제작업

데이터 호출

- Data frame 을 호출한후 몇가지 함수를 통해 데이터를 초기에 파악 할 수 있다.

이름	설명
df.head(n)	위에서 부터 n 개의 데이터를 보여준다.(default : 5)
df.tail(n)	아래 에서 부터 n 개의 데이터를 보여준다.(default : 5)
df.info()	데이터에 대한 구조적 정보를 알려 준다.
df.columns	컬럼명들을 리스트에 담아 보여준다.
df.describe()	통계요약
df['column name'].value_counts()	특정 컬럼의 빈도확인
df['column name'].unique()	특정 컬럼의 중복제거값

데이터호출과 정제작업

데이터 정제작업

- 수집된 데이터를 가져와 바로 분석을 하려고 보면 정제되지 못한 데이터가 많이 있다.
- 그 사례는 아래와 같으며 이를 해결하는 작업을 우리는 cleaning 이라고 부른다.

이름	설명
Empty data	데이터가 비어있는 경우(결측 치)
Wrong format	들어온 데이터가 기존 데이터와 형태가 맞지 않는 경우
Wrong data	들어오는 데이터의 범위를 크게 벗어난 경우(이상치)
Duplicate	중복된 데이터가 들어오는 경우

실습파일 : 05_cleaning.ipynb

04.

데이터 고급기술

데이터 고급기술

correlation

- 복잡한 데이터 속에서 이들 간의 상관 관계를 확인 할 수 있다.
- 상관관계가 강할 수록 데이터 간의 움직임에 관련성이 높다고 판단 할 수 있다.

```
import pandas as pd
root='/content/drive/MyDrive/pandas/data'
df = pd.read_csv(f'{root}/import.csv')
df.corr()
```

실습파일 : 06_correlation.ipynb

• 실행결과

	Duration	Pulse	Max pulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922717
Pulse	-0.155408	1.000000	0.786535	0.025121
Max pulse	0.009403	0.786535	1.000000	0.203813
Calories	0.922717	0.025121	0.203813	1.000000

데이터 고급기술

Map & lambda

- Map 은 data Frame 을 하나씩 뽑아내는 기능을 한다.
- Lambda 는 map 에서 꺼내 온 데이터를 하나씩 처리 할 때 사용 한다.
- Map 에서 처리된 내용은 새로운 데이터 프레임으로 반환 된다.

```
df['Calories'].map(lambda x: print(f'Calorie:{x}'))
```

실습파일 : 07_map.ipynb

```
df['OverCalories']=df['Calories'].map(lambda x: 'Over' if (x>1000) else 'Under')
```

실습파일 : 08_lambda.ipynb

데이터 고급기술

Pivot

- Pivot table 은 데이터의 두개 열을 각각 행과 열 인덱스로 펼쳐 놓은 것을 말한다.
- DB 에서든 엑셀 에서든 쉽지 않은 작업이다.
- Pandas 에서는 이런 피벗 테이블을 손쉽게 생성 할 수 있다.

도시	연도	인구	지역
서울	2010	9,631,482	수도권
서울	2015	9,762,546	수도권
부산	2010	3,393,191	경상권
부산	2015	3,448,737	경상권
인천	2010	263,203	수도권
인천	2015	2,890,451	수도권



	2010	2015
부산	3,393,191	3,448,737
서울	9,631,482	9,762,546
인천	263,203	2,890,451

데이터 고급기술



column row(index)

```
import pandas as pd
data = {
    "도시": ["서울", "서울", "서울", "부산", "부산", "부산", "인천", "인천"],
    "연도": ["2015", "2010", "2005", "2015", "2010", "2005", "2015", "2010"],
    "인구": [9904312, 9631482, 9762546, 3448737, 3393191, 3512547, 2890451, 263203],
    "지역": ["수도권", "수도권", "수도권", "경상권", "경상권", "경상권", "수도권", "수도권"]
}
df = pd.DataFrame(data)
```

```
new_df=df.pivot(index="도시", columns="연도", values="인구")
```

실습파일 : 09_pivot.ipynb

데이터 고급기술

✓ Group By

- Group by 는 특정한 기준으로 데이터들을 묶어서 집계 할 때 사용 한다.
- 평균, 크기 등을 구할 수 있다.
- Agg() 함수를 통해 다양한 집계를 할 수 있으며
- Apply() 함수를 이용하여 Series 와 DataFrame 을 하나씩 꺼내서 처리 할 수 있다.

city	fruits	price	quantity
부산	apple	100	1
부산	orange	200	2
부산	banana	250	3
부산	banana	300	4
서울	apple	150	5
서울	apple	200	6
서울	banana	400	7

```
group.agg(  
    fruit_count=('fruits', 'count'),  
    price_sum=('price', 'sum'),  
    qty_sum=('quantity', 'sum')  
)
```

실습파일 : 10_group_by.ipynb



city	fruit_count	price_sum	qty_sum
부산	4	850	10
서울	3	750	18

데이터 고급기술

Pivot Table

- Pivot_table() 은 pivot 명령어와 group by 명령어의 중간 성격을 갖는다.
- pivot()의 모든 기능을 포함 하면서 집계(aggregation) 기능이 추가된 더 강력한 기능이다.

```
df.pivot_table(  
    columns='연도',  
    index='도시',  
    values='인구',  
    fill_value=0,  
    margins=True,  
    aggfunc='mean',  
    margins_name='평균',  
)
```

실습파일 : 11_pivot_table.ipynb

옵션	설명
columns	열(column) index
index	행(row) index
values	분석할 index
fill_value	결측 값에 대한 대체 값
margins	True 는 집계 행/열을 추가한다.
aggfunc	집계할 함수(없으면 기본 mean)
margins_name	추가된 행/열의 이름(없으면 All 로 지정됨)

Summary



Series & DataFrame

- Series 모든 데이터 타입을 수용하는 1차원 리스트와 같다.
- Data frame 은 2차원 배열 형태의 데이터 이다.

Calories series		Duration series		Score series	
0	420	0	50	0	100
1	380	1	40	1	70
2	390	2	45	2	80

+

Calories series		Duration series		Score series	
0	420	0	50	0	100
1	380	1	40	1	70
2	390	2	45	2	80

+

Calories series		Duration series		Score series	
0	420	0	50	0	100
1	380	1	40	1	70
2	390	2	45	2	80

↓

	column				
	calories	duration	score		
0	420	50	100	row	
1	380	40	70		
2	390	45	80		

Summary

☑ Data Cleaning

- 수집된 데이터를 가져와 바로 분석을 하려고 보면 정제되지 못한 데이터가 많이 있다.
- 그 사례는 아래와 같으며 이를 해결하는 작업을 우리는 cleaning 이라고 부른다.

이름	설명
Empty data	데이터가 비어있는 경우(결측 치)
Wrong format	들어온 데이터가 기존 데이터와 형태가 맞지 않는 경우
Wrong data	들어오는 데이터의 범위를 크게 벗어난 경우(이상치)
Duplicate	중복된 데이터가 들어오는 경우

☑ Pivot & Group By

- Pivot table 은 데이터의 두개 열을 각각 행과 열 인덱스로 펼쳐 놓은 것을 말한다.
- Group by 는 특정한 기준으로 데이터들을 묶어서 집계 할 때 사용 한다.
- Pivot_table() 은 pivot()의 기능과 집계(aggregation) 기능이 추가된 강력한 기능이다.